

本次对话学习笔记：MCP 工具池、System Prompt 缓存与代码审核

整理范围：call_tool / handler(**args) / assemble_tool_pool / MCPClient.call_tool / system prompt 组装 / 代码审核 / MCP 三条指令理想流程

核心结论

这次对话的主线是：模型并不是直接执行工具，而是输出 tool_use；Python agent loop 根据工具名找到 handler，并用 handler(**args) 把模型给出的 input 字典拆成关键字参数。MCP 接入后，外部工具会被统一加前缀，例如 mcp_docs_search，从而和内置工具进入同一个工具池。

- 当前代码的设计方向是对的：内置工具 + MCP 工具统一注册；system prompt 从 context 动态组装；工具执行通过 handlers 映射。
- 当前代码的实现还存在几个确定性错误：把 assemble_tool_pool 函数当成 tuple 下标使用、mcp_clients.key() 写错、execute_tool 错用 assemble_tool_pool、子 agent 的 cwd 参数没有和底层工具签名对齐。
- docs MCP 的 “Found 3 results” 不是实际搜索结果，而是 mock server 中 search lambda 硬编码返回的字符串。

生成时间：2026-07-06；依据：本轮对话与用户上传代码文件 Pasted text.txt。

目录

- 1. handler(**args)：工具参数字典如何进入函数
- 2. assemble_tool_pool：为什么要返回 tools 和 handlers
- 3. MCPClient.call_tool：filesystem_client 为什么能 .call_tool()
- 4. assemble_system_prompt 与 get_system_prompt 是否重复
- 5. 当前代码审核：确定性 bug 与修复建议
- 6. 三条 MCP 指令的理想执行流程
- 7. “ Found 3 results ” 的来源
- 8. 总体心智模型与修复顺序

1. handler(**args)：工具参数字典如何进入函数

你最开始问的代码是：

```
def call_tool(self, tool_name: str, args: dict) -> str:
    handler = self._handlers.get(tool_name)
    if not handler:
        return f"MCP error: unknown tool '{tool_name}'"
    try:
        return handler(**args)
    except Exception as e:
        return f"MCP error: {e}"
```

这里的 args 是一个普通字典，通常来自模型 tool_use 的 input。例如模型请求读取文件时，input 可能是 {"file_path": "README.md"}。

handler(**args) 是 Python 的关键字参数解包：把字典的 key 当作形参名，把 value 当作实参值。

```
args = {"file_path": "README.md"}
handler(**args)

# ■■■■
handler(file_path="README.md")
```

因此它有一个强约束：args 的 key 必须和真实工具函数的形参名一致。如果函数定义是 run_bash(command)，但 args 是 {"cmd": "ls"}，就会报 unexpected keyword argument。



2. assemble_tool_pool：为什么要返回 tools 和 handlers

你后面问的 assemble_tool_pool 本质是“工具池合并器”。它把本地内置工具和所有已连接 MCP server 的工具合并成两个结构：

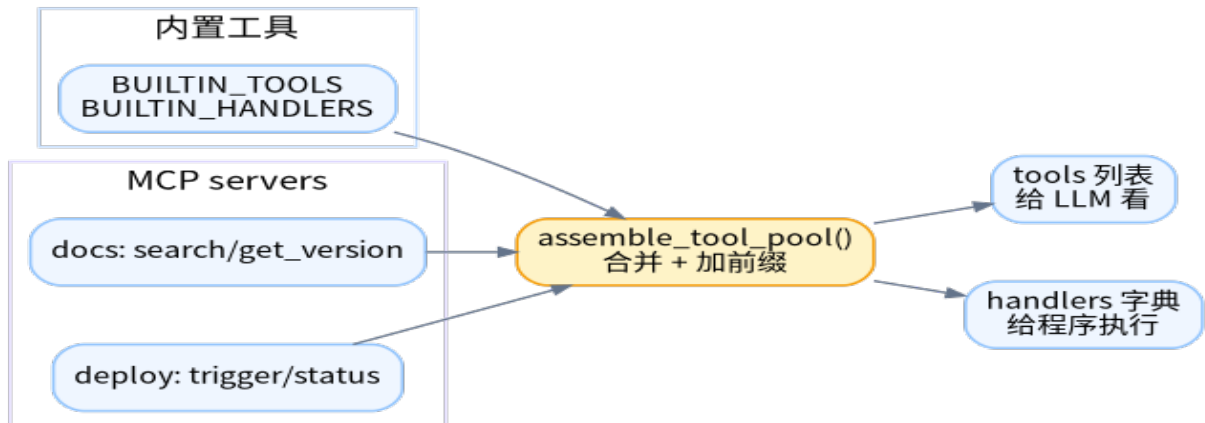
返回值	用途	给谁用
tools: list[dict]	工具说明书，包括 name、description、input_schema	给 LLM 看，让模型知道能调用哪些工具
handlers: dict	工具名 -> Python 可调用对象	给 agent loop 执行工具时查表用

内置工具本来就有名字，比如 read_file、write_file。MCP 工具来自不同 server，可能重名，所以必须加前缀。

```
prefixed = f"mcp__{safe_server}__{safe_tool}"

# ■■■■
# docs.search -> mcp__docs__search
# deploy.status -> mcp__deploy__status
```

这一步的意义是避免冲突：docs server 可能有 search，GitHub server 也可能有 search。加上 server 前缀后，模型看到的工具名是唯一的。



3. MCPClient.call_tool：为什么 client 能 .call_tool()

你问“filesystem_client 为什么能 .call_tool”。答案是：变量名不重要，重要的是这个对象所属的类定义了 call_tool 方法。你的代码中 MCPClient 类就定义了 call_tool。

```
class MCPClient:
    def __init__(self, name: str):
        self.name = name
        self.tools: list[dict] = []
        self._handlers: dict[str, callable] = {}

    def call_tool(self, tool_name: str, args: dict) -> str:
        handler = self._handlers.get(tool_name)
        if not handler:
            return f"MCP error: unknown tool '{tool_name}'"
        try:
            return handler(**args)
        except Exception as e:
            return f"MCP error: {e}"
```

类比：字符串能调用 text.upper()，不是因为变量名叫 text，而是因为 str 类有 upper 方法。同理，mcp_client 能调用 call_tool，是因为它是 MCPClient 实例。

MCP 工具执行链可以理解成两层 handler：外层 handler 负责把 mcp__docs__search 转发给 docs client；内层 handler 才是真正的 search 函数。

```
# ■■■agent handlers
handlers["mcp__docs__search"] = adapter

# adapter ■■■■
docs_client.call_tool("search", {"query": "something"})

# ■■■docs_client._handlers
"search": lambda query: f"[docs] Found 3 results for '{query}'"
```

4. assemble_system_prompt 与 get_system_prompt 是否重复

这两个函数不算完全重复，它们的职责不同：

函数	职责	关键词
assemble_system_prompt(context)	真正拼接 system prompt 内容：identity、workspace、tools、skills、memory、MCP server 信息	构造器
get_system_prompt(context)	根据 context 生成缓存 key；如果 context 没变就复用旧 prompt；否则重新 assemble	缓存包装器

但你当前代码有一个设计隐患：真实 section 拼接在 assemble_system_prompt 里，日志 loaded 列表却在 get_system_prompt 里手动维护。两边容易不同步。

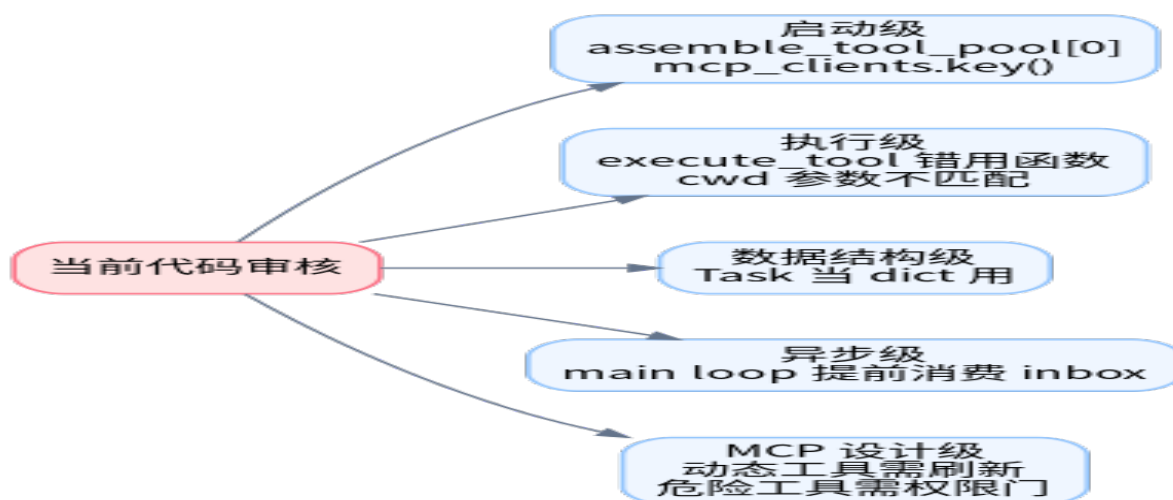
推荐改法是让 assemble_system_prompt 同时返回 prompt 和 loaded_sections，这样日志永远反映真实注入内容。

```
def assemble_system_prompt(context: dict) -> tuple[str, list[str]]:
    sections, loaded = [], []
    # ■ append ■■ section■■■■ loaded.append(section_name)
    return "

".join(sections), loaded
```

5. 当前代码审核：确定性 bug 与修复建议

下面这些是本轮审核中确认的重点问题。它们不是风格问题，而是会导致启动失败、工具执行失败或异步消息处理不符合设计预期。



严重级别	问题位置	当前写法/现象	推荐修复
致命	update_context	list(assemble_tool_pool[0]) 把函数当 tuple 下标使用	先 tools, _ = assemble_tool_pool(), 再 [t["name"] for t in tools]
致命	assemble_system_prompt	mcp_clients.key() 拼写错误	改成 mcp_clients.keys(); 更好是从 context["mcp_servers"] 读取
致命	execute_tool	assemble_tool_pool[1].get(...) 同样把函数当 tuple	直接用 agent_loop 里已有的 handlers; 或 execute_tool(block, handlers)
致命	子 agent worktree	run_bash/read/write 被传 cwd=, 但原函数不接受 cwd	给底层函数增加 cwd 参数, 或 wrapper 不传 cwd
致命	idle_poll	task.get 和 task["worktree"] 把 dataclass 当 dict 用	改成 task.worktree
结构隐患	main loop 尾部 inbox	consume_lead_inbox 读空 inbox, 但不立刻跑 agent_loop	删除该段, 让 inbox_processor_loop 只提醒、check_inbox 唯一消费

最小修复片段 1：update_context

```
def update_context(context: dict, messages: list) -> dict:
    memory_index = read_memory_index()
    relevant_memories = load_memories(messages) if messages else ""

    tools, _ = assemble_tool_pool()

    return {
        "enabled_tools": [tool["name"] for tool in tools],
        "workspace": str(WORKDIR),
        "skills": list_skills(),
        "memory_index": memory_index,
        "memories": relevant_memories,
        "mcp_servers": list(mcp_clients.keys()),
    }
```

最小修复片段 2：system prompt 中的 MCP server

```
mcp_names = context.get("mcp_servers", [])
if mcp_names:
    sections.append(f"Connected MCP servers: {' '.join(mcp_names)}")
```

最小修复片段 3：工具执行不要再错误调用 `assemble_tool_pool[1]`

你在 `agent_loop` 里已经有 `tools, handlers =`

`assemble_tool_pool()`，因此普通同步工具执行处可以直接用当前工具表中的 `tool_handler`。

```
tool_handler = handlers.get(block.name)
if not tool_handler:
    output = f"Unknown tool: {block.name}"
else:
    output = tool_handler(**block.input)
```

如果要保留 `execute_tool`，建议改成显式传 `handlers`：

```
def execute_tool(block, handlers: dict) -> str:
    handler = handlers.get(block.name)
    if handler:
        return handler(**block.input)
    return f"Unknown tool: {block.name}"
```

最小修复片段 4：Task dataclass 不要当 dict

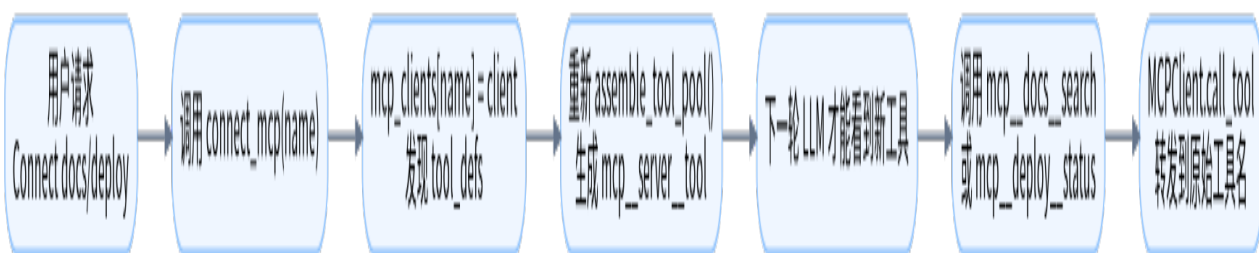
```
if task.worktree:
    wt_path = WORKTREES_DIR / task.worktree
    wt_info = f"
Work directory: {wt_path}"
```

最小修复片段 5：inbox 的单一消费入口

推荐原则：`inbox_processor_loop` 只检测并注入通知；`check_inbox` 工具才真正 `consume_lead_inbox`；`main loop` 尾部不要再主动读空 `lead inbox`。

6. 三条 MCP 指令的理想执行流程

严格按当前代码运行会先遇到前面的 bug；下面是假设这些 bug 已经修复后的理想行为。



指令 1：Connect to the docs MCP server and search for something

- 第 1 轮：LLM 只能看到内置 `connect_mcp`，于是调用 `connect_mcp(name="docs")`。
- Python 注册 docs client 到 `mcp_clients`，并发现 `search`、`get_version`。
- `agent loop` 发现本轮调用过 `connect_mcp`，于是重新 `assemble_tool_pool`。
- 第 2 轮：LLM 才能看到 `mcp_docs_search`，然后调用 `mcp_docs_search(query="something")`。
- mock handler 返回：`[docs] Found 3 results for 'something'`。

指令 2：Connect to the deploy server and trigger a deployment

- 第 1 轮：调用 connect_mcp(name="deploy")，注册 deploy server。
- 刷新工具池后出现 mcp__deploy__trigger 和 mcp__deploy__status。
- trigger 工具需要 service 参数，而且描述中标记为 destructive。理想 agent 不应乱猜 service，而应询问要部署哪个 service，或要求明确确认。

指令 3：Connect both servers — what tools are now available?

- 可以同一轮调用 connect_mcp(name="docs") 和 connect_mcp(name="deploy")。
- 刷新后 MCP
工具应包括：mcp__docs__search、mcp__docs__get_version、mcp__deploy__trigger、mcp__deploy__status。
- 最终回答应区分内置工具和 MCP 新增工具。

连接后 server	新增 MCP 工具	参数
docs	mcp__docs__search	query: string
docs	mcp__docs__get_version	无参数
deploy	mcp__deploy__trigger	service: string
deploy	mcp__deploy__status	service: string

7. “ Found 3 results ” 的来源

你问“哪里找到了三个 result”。答案是：没有真的找。这个结果来自 mock server 的硬编码 lambda。

```
def _mock_server_docs():
    client = MCPClient("docs")
    client.register(
        tool_defs=[...],
        handlers={
            "search": lambda query: f"[docs] Found 3 results for '{query}'",
            "get_version": lambda: "[docs] API v2.1.0",
        })
    return client
```

所以不管 query 是什么，只要调用 docs.search，都会返回“Found 3 results”。这只是教学 mock，用来模拟外部 MCP 搜索工具的响应。真实版本需要把 search handler 替换为真正的数据源，例如本地文档索引、API、数据库或向量检索。

8. 总体心智模型与修复顺序

这次对话可以压缩成一个主链路：

```
LLM ■■ tools schema
↓
LLM ■■ tool_use(name, input)
↓
agent loop ■ handlers ■■■
↓
handler(**input) ■■
↓
■■■■ tool_result ■■ messages
↓
LLM ■■■■■■■■ stop_reason != tool_use
```

MCP 只是给这条链路增加了一层“外部工具发现 + 名字前缀 + 转发适配器”。

建议修复顺序

- 先修启动级：update_context 里的 assemble_tool_pool[0]，assemble_system_prompt 里的 mcp_clients.key()。
- 再修执行级：execute_tool 不要再 assemble_tool_pool[1]；工具执行用当前 handlers。
- 再修子 agent/worktree：run_bash/run_read/run_write 是否支持 cwd，参数名是否和 schema 对齐。
- 再修异步 inbox：统一消费入口，避免 main loop 读空消息但不马上处理。
- 最后补强 MCP 权限：对 destructive MCP tools 增加确认或审批。

最终判断：你的整体架构方向是正确的，真正的问题不是“概念错”，而是 s10-s19 多阶段合并后，部分旧调用点没有同步更新到新的动态工具池模型。